

МФТИ
Алгоритмы и структуры данных I, осень 2024
Программа экзамена

Всюду, где уместно и не сказано иное, пункт программы подразумевает формулировку решаемой задачи, описание алгоритма, доказательство его корректности и анализ асимптотики. Факты, не требующие доказательств на экзамене, помечены «б/д».

1. RAM-модель вычислений: допустимые операции. Измерение времени работы алгоритмы в терминах асимптотики: пример.
2. Асимптотические обозначения: O , Ω , Θ . Независимость от стартового индекса. Примеры.
3. Решение рекурренты $T(n) = 2T(n/2) + O(n)$.
4. Проверка вхождения числа в отсортированный массив: бинарный поиск.
5. Задача сортировки. Две постановки, их эквивалентность.
6. Доказательство формулы: $\log(n!) = \Theta(n \log n)$.
7. Нижняя оценка на число сравнений в сортировке сравнениями.
8. Сортировка слиянием (Merge Sort).
9. Подсчёт числа инверсий в массиве.
10. Быстрая сортировка (Quick Sort). Асимптотика — б/д.
11. Поиск k -й порядковой статистики с выбором случайного пивота (Quick Select). Асимптотика — б/д.
12. Алгоритм поиска k -й порядковой статистики за $O(n)$, где n — длина массива.
13. Алгоритм быстрой сортировки за $O(n \log n)$, где n — длина массива.
14. Стабильная сортировка подсчётом. Сортировка пар чисел.
15. Цифровая сортировка (LSD).
16. Бинарная куча: определение и представление в массиве. Требование кучи.
17. Операции `siftUp` и `siftDown` в бинарной куче с доказательством корректности.
18. Выражение `insert`, `getMin`, `extractMin` и `decreaseKey` через `siftUp` и `siftDown` в бинарной куче.
19. Построение кучи (`heapify`) за линейное время (сходимостью ряда можно пользоваться б/д).
20. Сортировка кучей с привлечением $O(1)$ дополнительной памяти (Heap Sort). Несуществование кучи (основанной на сравнениях), обрабатывающей `insert` и `extractMin` за $O(1)$.
21. Удаление из кучи по указателю.
22. Удаление из кучи по значению.
23. Биномиальное дерево, биномиальная куча: определение. Преимущество перед двоичной кучей.
24. Операции `merge`, `insert`, `getMin`, `extractMin` и `decreaseKey` в биномиальной куче.
25. Амортизационный анализ, учётное время работы: определение.
26. Структура данных «вектор» (динамический массив), реализация на массиве и оценка асимптотики.
27. Односвязный, двусвязный списки. Добавление, удаление элементов.
28. Фибоначчиева куча. Реализация запросов `merge`, `insert`, `getMin`, `extractMin`. Оценка времени работы через $D(n)$ — максимальную степень вершины в куче из n элементов.
29. Фибоначчиева куча. Реализация `decreaseKey`.
30. Фибоначчиева куча: асимптотика функции $D(n)$ (максимальная степень вершины в куче из n элементов). Доказательство асимптотики всех операций.
31. Структура данных «стек»: реализация списком.
32. Проверка правильности скобочной последовательности с несколькими типами скобок.
33. Поддержка минимума в стеке.
34. Структура данных «очередь»: реализация списком.
35. Реализация очереди на двух стеках. Асимптотика операций. Поддержка минимума в очереди.
36. Sparse Table: модельная задача, построение за $O(n \log n)$, ответ на запрос за $O(1)$.

37. Дерево отрезков: модельная задача. Обработка запросов с доказательством времени работы.
38. Дерево отрезков: двоичный спуск, поиск подотрезка с суммой $\geq x$.
39. Дерево отрезков, отложенные операции: присваивание на отрезке, операция **push**.
40. Количество чисел на отрезке, значения которых не превосходят x : Fractional Cascading.
41. Динамическое дерево отрезков.
42. Сжатие координат для дерева отрезков.
43. Персистентный массив.
44. Персистентный стек.
45. Персистентное дерево отрезков.
46. Количество чисел на отрезке, значения которых не превосходят x : решение с персистентным деревом отрезков.
47. Поиск k -й порядковой статистике на отрезке: решение с персистентным деревом отрезков и параллельным спускам по двум его версиям.
48. Дерево поиска: определение и операции (без реализации) **find**, **insert**, **erase**, а также опциональные **merge** и **split**. Запросы, использующие порядок на элементах.
49. Наивное дерево поиска, обработка операций.
50. AVL-дерево: определение. Оценка глубины AVL-дерева на n вершинах.
51. Устранение дисбаланса в AVL-дереве.
52. AVL-дерево: реализация операций **find**, **insert** и **erase**.
53. Splay-дерево: определение и практическая значимость.
54. Splay-дерево: операции **zig**, **zig-zig** и **zig-zag**, операция **splay**.
55. Метод потенциалов.
56. Амортизированное время работы операции **splay** с помощью метода потенциалов.
57. Splay-дерево: реализация **insert**, **erase** и **find**, связь с операцией **splay**, оценка времени работы (б/д).
58. Splay-дерево: реализация **merge** и **split**, связь с операцией **splay**, оценка времени работы (б/д).
59. Дерево поиска по неявному ключу: в массиве вставить, удалить, изменить элемент, нахождение суммы на отрезке.
60. Декартово дерево: определение и теорема о глубине (б/д). Построение за линейное время (при условии, что ключи отсортированы).
61. Реализация операций **merge** и **split** в декартовом дереве.
62. Выражение **insert** и **erase** в декартовом дереве через **merge** и **split**.
63. В-дерево: определение и практическая значимость. Реализация операции **find**.
64. Оценка глубины В-дерева на n ключах при фиксированном параметре t .
65. Реализация операции **insert** в В-дереве.
66. Реализация операции **erase** в В-дереве.
67. Красно-чёрное дерево: определение.
68. Оценка глубины красно-чёрного дерева на n ключах.
69. [Можно пользоваться официальной шпаргалкой с разбором случаев] Реализация операции **insert** в красно-чёрном дереве.
70. [Можно пользоваться официальной шпаргалкой с разбором случаев] Реализация операции **erase** в красно-чёрном дереве.
71. Сравнительный анализ различных реализаций дерева поиска: наивное, AVL-, splay-, В-, декартово и красно-чёрное дерево.
72. Хеш-функции. Коллизии. Универсальное и k -независимое семейства хеш-функций. Примеры таких семейств.
73. Хеш-таблицы с цепочками. Реализация операций **find**, **insert**, **erase**. Процедура **rehash**.
74. Совершенное хеширование.
75. Хеш-таблицы с открытой адресацией. Реализация операций **find**, **insert**, **erase**. Процедура **rehash**. Теорема о времени работы для линейного пробирования, двойного хеширования.

- 76. Фильтр Блума. Алгоритм и оптимальные значения параметров (б/д).
- 77. Фильтр кукушки (б/д).
- 78. Skip List. Операции `find`, `insert`, `erase`. Асимптотика — б/д.